Designing a Fibonacci Accelerator

Authors:
WhisperingRock

Class
CPE490: Open-Source ASIC Design
October 27, 2026

Contents

Contents 1

1 Introductions **2**

2 Procedure **3**

3 Design **4**

4 Simulation Testing **10**
4.1 Brent-Kung 'Parallel Prefix' Adder 10
4.2 Fibonacci Accelerator 10

5 Appendix **11**
5.1 Code 11
5.1.1 RTL 11
5.1.2 Simulation SVA Testing 16
5.2 Boneyard 18

6 References **19**

Chapter 1

Introductions

In this lab, the task was to create a Fibonacci accelerator. Before going further in the paper, I must first describe the Fibonacci sequence and what makes a design an *accelerator*.

First, let's discuss the Fibonacci sequence using a recursive formula.

$$\boldsymbol{F_{n+2} = F_{n+1} + F_n} \text{ where } \boldsymbol{F_0 = 0} \text{ and } \boldsymbol{F_1 = 1}$$

In our discussion in class, another relationship emerged when comparing multiple terms.

Consider:

- Let $F_n = a$
- Let $F_{n+1} = b$
- $F_{n+2} = a + b$
- $F_{n+3} = a + 2b$
- $F_{n+4} = 2a + 3b$
- $F_{n+5} = 3a + 5b$

If given two terms of the sequence, we can potentially calculate the next four terms from the initial two. This unique way to describe terms has potential for a speed-up if we can utilize shifters and adders with the motivation to calculate many terms in a single cycle.

This motivation to increase throughput or performance over a CPU, through the use of configurable logic, is termed as an *accelerator*. For this lab, our accelerator will calculate more than one term of the Fibonacci sequence per clock cycle.

Lastly, another motivation of this lab is to become familiar with open-source tooling set.

Chapter 2

Procedure

The workflow of this lab, which is largely focused on design before synthesis, will be the following

1. LINTing based off of many tools s.a. Verilator, Icarus, GL, ...
2. Simulation of test scripts on Verilator, then Icarus Verilog
3. BMC verification
4. K-Induction formal verification (optional)

Chapter 3

Design

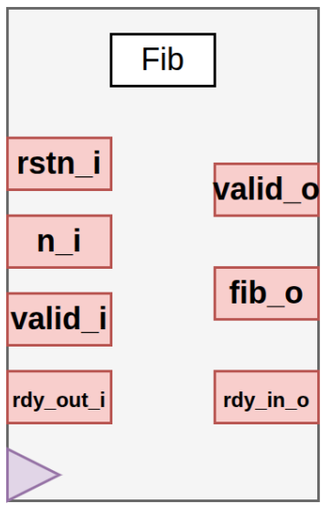


Figure 3.1: Fibonacci Top Module

The top-level module has a data and control signals:

- **clk_i**: clock (posedge is captured).
- **rstn_i**: active-low synchronous reset.
- **valid_i**: indicates when input n_i can be read.
- **n_i**: requested term of fib sequence.
- **rdy_out_i**: indicates when downstream has read this modules output (and module can begin processing a new sequence).

- **valid_o**: indicates module's output is ready to be read.
- **rdy_on_o**: indicates module is ready for a new request (n_i).
- **fib_o**: modules output (F_n).

In order to capture the functionality of these top-level signals, an FSM was used:

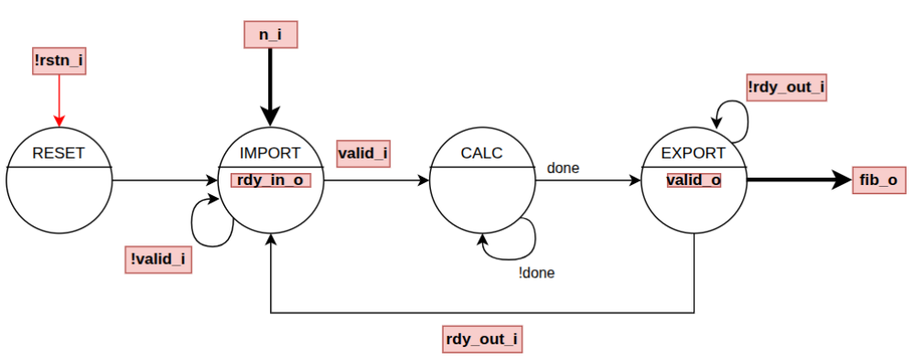


Figure 3.2: FSM of Fibonacci Accelerator

From here, lets take a closer look into the CALC stage of the FSM, where most of our accelerator logic comes into play. An attentive reader will notice that there are only two adders and a single shifter: there is a considerable amount of adders required to further calculate F_{n+4} and F_{n+5}. Since this must be implemented in ASIC where wires and hardware are not free lunches, this designer opted to take 2 cycles to calculate 4 terms of the fib sequence. This will keep the area smaller while producing two elements per cycle (a +2x increase over a CPU).

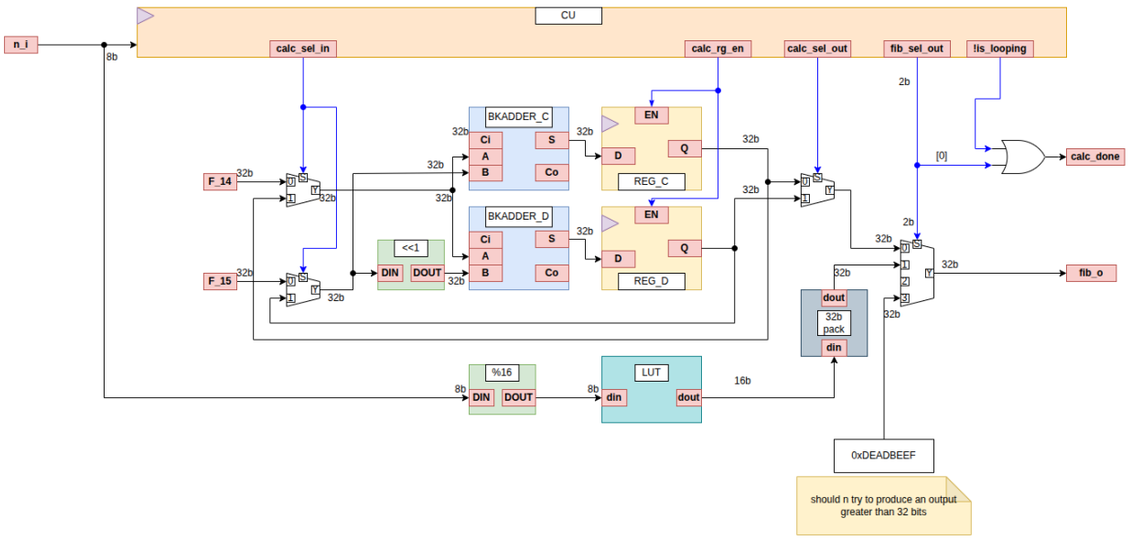


Figure 3.3: Computation Portion of Fib Accelerator

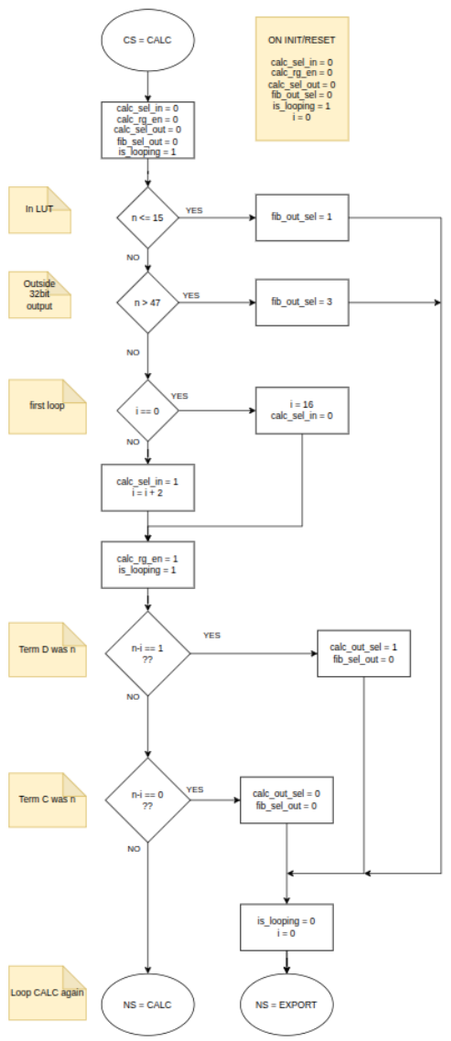
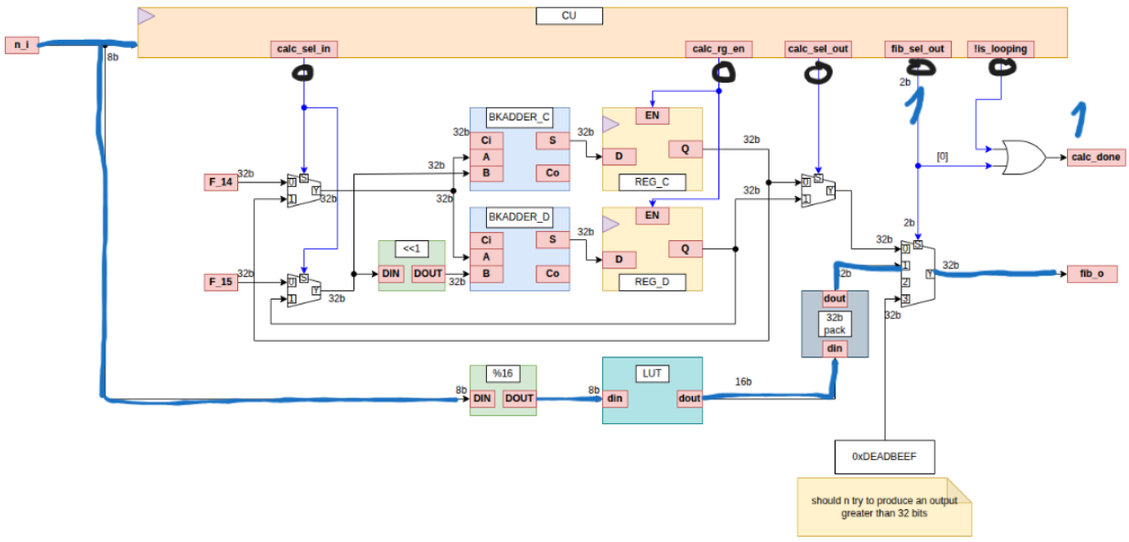


Figure 3.4: Control Flow of CALC State

To properly explain the design, let's trace through various values of n.



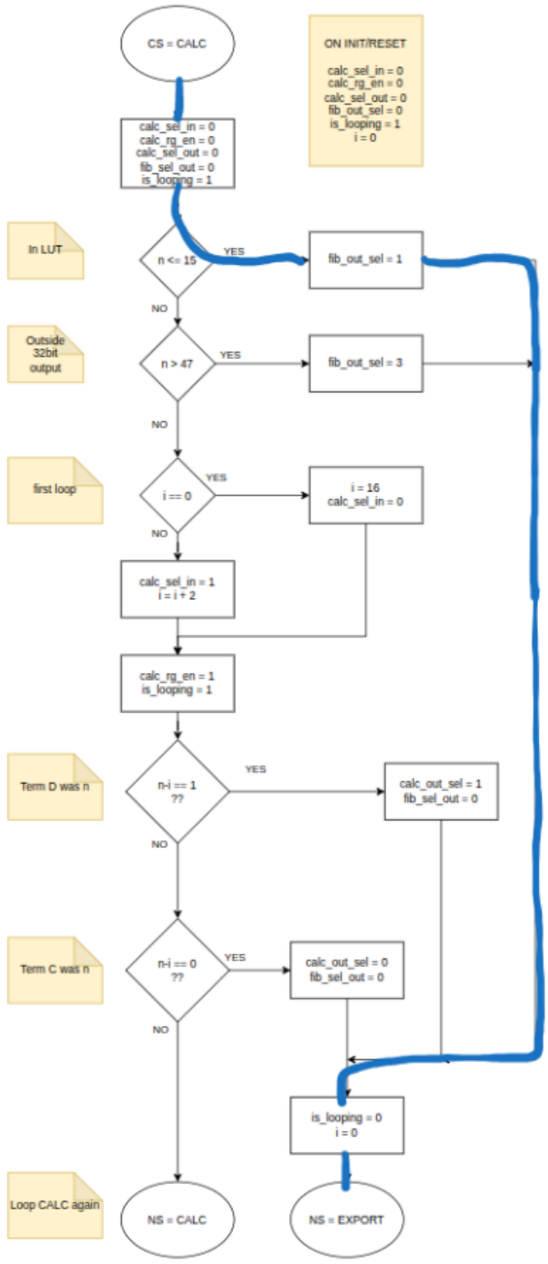
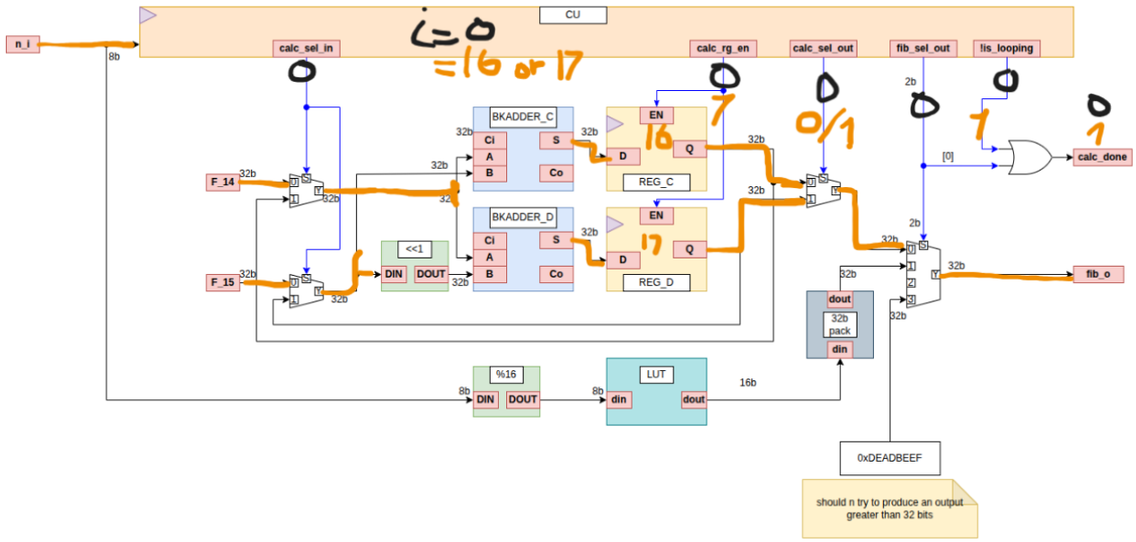


Figure 3.5: $n \leq 15$ uses the LUT



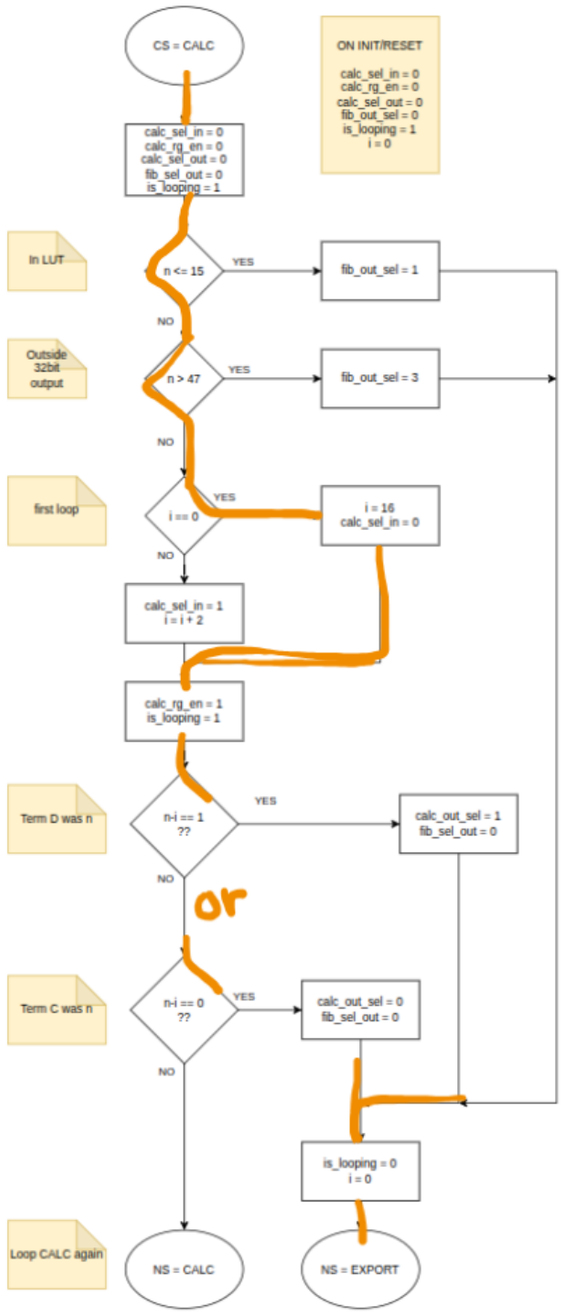


Figure 3.6: $n = 16$ or $n = 17$ calculates but doesn't loop

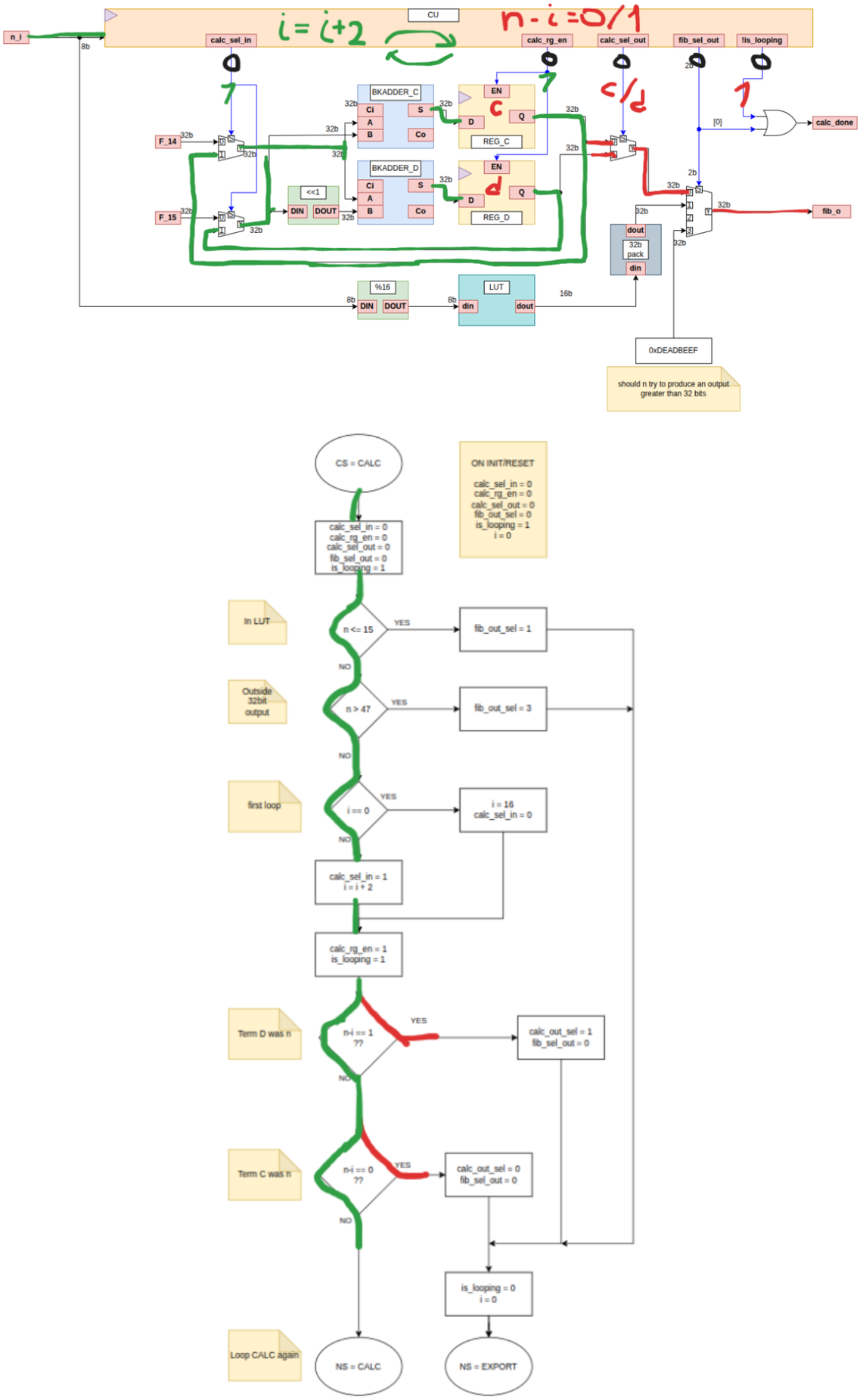


Figure 3.7: $n \geq 18$ and $n \leq 47$ calculates using loops

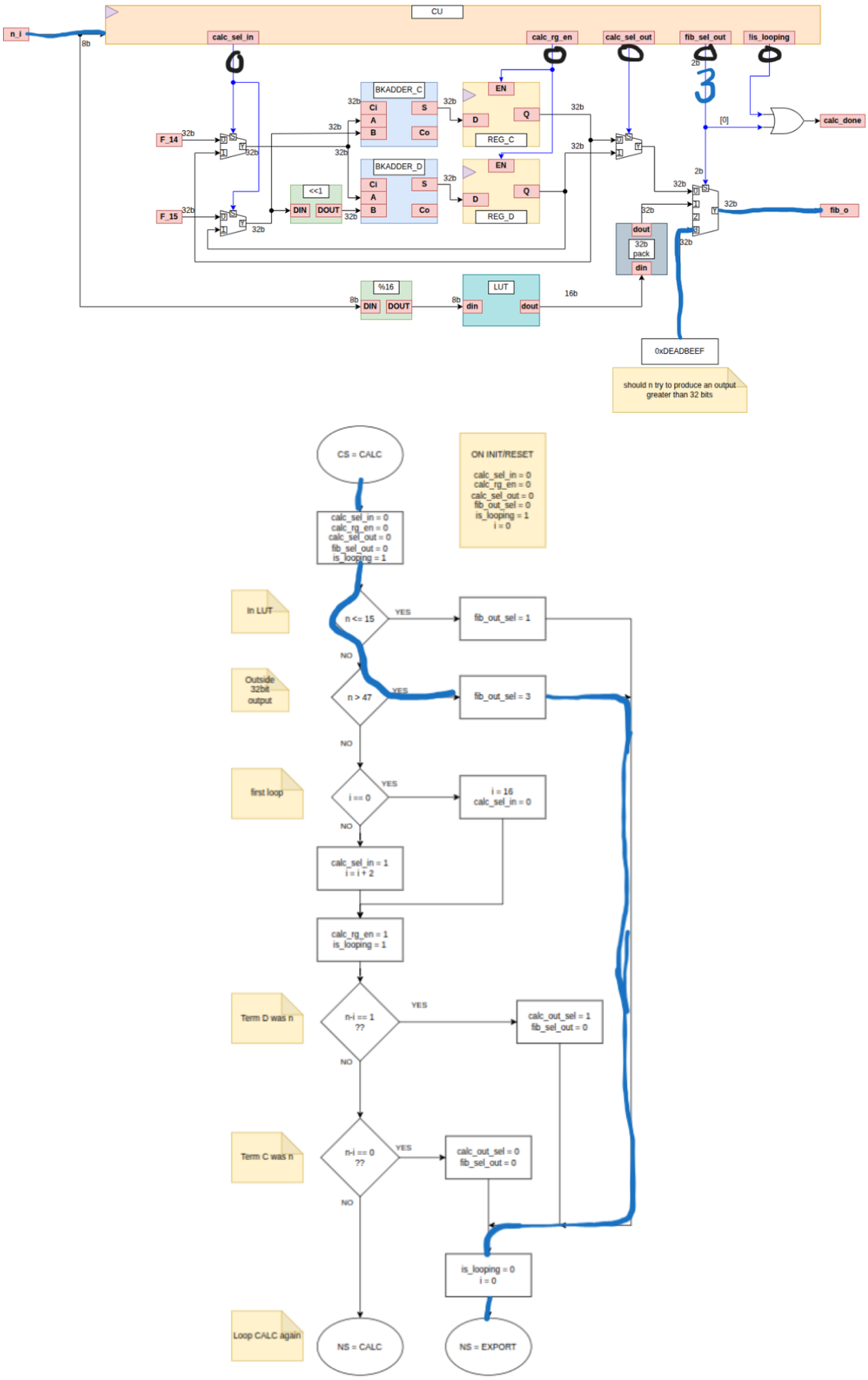


Figure 3.8: $n > 47$ defaults to avoid output overflow

Chapter 4

Simulation Testing

4.1 Brent-Kung 'Parallel Prefix' Adder

The BKAdder was studied and implemented as a dynamic model, only verified using formal verification methods BMC and K-Induction. The rtl and verification can be seen in the same file 5.2.

Test Type	Result
BMC	✓
Induction	✓

Table 4.1: Verification Methods for BKAdder

4.2 Fibonacci Accelerator

The accelerator was initially verified using immediate and concurrent simulations on verilator (not icarus because SVA is not supported). The simulation file 5.4 tests:

- **Concurrents**
  - **reset_import**: confirms the module comes out of RESET, enters the IMPORT state, and signals the upstream that it is ready to accept an input n.
  - **import_export**: confirms the model is not holding the output steady while prompting the upstream for a new input n.
  - **hold_fib**: confirms the output `fib_o` was steady as long as our module is finished calculating and the downstream has not confirmed an output read.
- **Immediates**
  1. Test $n \in [0, 47]$ outputs the correct values sequentially.
  2. Test $n \in [48, 255]$ outputs an error code because output overflow is certain.

Test Type	Result
Verilator (SVA) Concurrents	✓
Verilator (SVA) Immediates	✓
Icarus (SVA) Immediates	✓
BMC	✓
Induction	✗TODO

Table 4.2: Verification Methods for BKAdder

Overall, the design works well but further verification would make this author feel better about taping on silicon. For now, these desired will be placed on hold til the design is viewed with LibreLane.

Chapter 5

Appendix

5.1 Code

5.1.1 RTL

Listing 5.1: Mux

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 07/17/2026 04:24:06 PM
// Design Name:
// Module Name: Mux_N
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//                          din[0] occupies the LSB WIDTH slot
//////////////////////////////////////////////////////////////////////////////////


module Mux_N
#(
    parameter WIDTH     = 32,
    parameter INPUTS    = 4,
    parameter SEL_WIDTH = (INPUTS <= 1) ? 1 : $clog2(INPUTS)
)
(
    input logic     [INPUTS-1:0][WIDTH-1:0] DIN,
    input logic     [SEL_WIDTH-1:0]         SEL,
    output logic    [WIDTH-1:0]             DOUT
);

    always_comb begin

        // ~~ default ~~
        DOUT = '0;

        if(int'(SEL) < INPUTS) begin
            DOUT = DIN[SEL];
        end
    end

endmodule
```

Listing 5.2: Brent-Kung 'Parallel Prefix' Adder

```
`timescale 1ns / 1ps

module BKAdder
#( parameter int WIDTH = 32 )
(
    input  logic [WIDTH-1:0] a,
    input  logic [WIDTH-1:0] b,
    input  logic             cin,
    output logic [WIDTH-1:0] sum,
    output logic             cout
);

    // ~~~~ local vars ~~~~
    // Keep at least one tree level so WIDTH=1 is supported.
    localparam int LEVELS = (WIDTH <= 1) ? 1 : $clog2(WIDTH);
    localparam int N      = 1 << LEVELS;
    localparam int STAGES = 2*LEVELS - 1;

    wire [N-1:0] a_pad = {{(N-WIDTH){1'b0}}, a};
    wire [N-1:0] b_pad = {{(N-WIDTH){1'b0}}, b};

    // Prefix arrays: stage 0 contains each bit's individual terms.
    wire [N-1:0] P [0:STAGES];
    wire [N-1:0] G [0:STAGES];

    assign P[0] = a_pad ^ b_pad;
    assign G[0] = a_pad & b_pad;
```

```
28
29      // ~~~~ Up-sweep: combine adjacent groups ~~~~
30      for (genvar s = 0; s < LEVELS; s++) begin : gen_up
31          localparam int SPAN = 1 << s;
32          localparam int PREV = s;
33          localparam int NEXT = s + 1;
34
35          for (genvar i = 0; i < N; i++) begin : gen_bit
36              if (((i + 1) % (2*SPAN)) == 0) begin : gen_combine
37                  assign G[NEXT][i] =
38                      G[PREV][i] | (P[PREV][i] & G[PREV][i-SPAN]);
39                  assign P[NEXT][i] =
40                      P[PREV][i] & P[PREV][i-SPAN];
41              end else begin : gen_pass
42                  assign G[NEXT][i] = G[PREV][i];
43                  assign P[NEXT][i] = P[PREV][i];
44              end
45          end
46      end
47
48      // ~~~~ Down-sweep: fill in the remaining prefixes ~~~~
49      for (genvar t = 0; t < LEVELS-1; t++) begin : gen_down
50          localparam int D    = LEVELS - 2 - t;
51          localparam int SPAN = 1 << D;
52          localparam int PREV = LEVELS + t;
53          localparam int NEXT = PREV + 1;
54
55          for (genvar i = 0; i < N; i++) begin : gen_bit
56              if ((i >= 3*SPAN - 1) &&
57                  (((i - (3*SPAN - 1)) % (2*SPAN)) == 0)) begin : gen_combine
58                  assign G[NEXT][i] =
59                      G[PREV][i] | (P[PREV][i] & G[PREV][i-SPAN]);
60                  assign P[NEXT][i] =
61                      P[PREV][i] & P[PREV][i-SPAN];
62              end else begin : gen_pass
63                  assign G[NEXT][i] = G[PREV][i];
64                  assign P[NEXT][i] = P[PREV][i];
65              end
66          end
67      end
68
69      wire [N-1:0] carry;
70
71      // ~~~~ collect sum and carry out bit ~~~~
72      for (genvar k = 0; k < WIDTH; k++) begin : gen_sum
73          if (k == 0) begin : gen_first
74              assign carry[k] = cin;
75          end else begin : gen_other
76              assign carry[k] =
77                  G[STAGES][k-1] | (P[STAGES][k-1] & cin);
78          end
79
80          assign sum[k] = P[0][k] ^ carry[k];
81      end
82
83      assign cout =
84          G[STAGES][WIDTH-1] | (P[STAGES][WIDTH-1] & cin);
85
86
87      `ifdef FORMAL
88          always @(*) begin
89              assert( (a + b + cin) == {cout, sum});
90          end
91      `endif
92
93  endmodule
```

Listing 5.3: Fibonacci Accelerator

```
1   `timescale 1ns / 1ps
2
3   /*
4   * Purpose : creating a fibonacci seq
5   *
6   * Inputs :
7   *       clk_i       : clock w/ pos active edge
8   *       rstn_i      : active low synch reset (resets on zero)
9   *       n_i         : nth fib sequence to stop at
10  *       valid_i     : driven high when n_i is ready to be read
11  *       rdy_out_i   : driven high when module's output has been read
12  *
13  * Returns :
14  *       rdy_in_o    : driven high by module to request new input data
15  *       fib_o       : calculated Fibonacci[n]
16  *       valid_o     : driven high by module when output data is ready
17  */
18
19  module fib
20  #(
21      parameter INBITS    = 8,
22      parameter OUTBITS   = 32
23  )
24  (
25      input logic                 clk_i,
26      input logic                 rstn_i,
27      input logic [INBITS-1:0]    n_i,
28      input logic                 valid_i,
29      input logic                 rdy_out_i,
30
31      output logic                rdy_in_o,
32      output logic [OUTBITS-1:0]  fib_o,
33      output logic                valid_o
34  );
35
36
37      // ~~~~ local vars ~~~~
38      typedef enum logic [1:0] {RESET, IMPORT, CALC, EXPORT} e_state;
```

```
    e_state CS, NS;

    localparam logic [7:0] LUTMAX       = 8'h0F; //15
    localparam logic [7:0] NMAX         = 8'h2F; //47
    localparam logic [31:0] FIBMAX      = 32'd2971215073;
    localparam logic [31:0] ERR_CODE    = 32'hFFFF_FFFF;

    function logic [15:0] LUT (input logic [3:0] idx);
        case(idx)
            4'd0:       LUT = 16'd0;
            4'd1:       LUT = 16'd1;
            4'd2:       LUT = 16'd1;
            4'd3:       LUT = 16'd2;
            4'd4:       LUT = 16'd3;
            4'd5:       LUT = 16'd5;
            4'd6:       LUT = 16'd8;
            4'd7:       LUT = 16'd13;
            4'd8:       LUT = 16'd21;
            4'd9:       LUT = 16'd34;
            4'd10:      LUT = 16'd55;
            4'd11:      LUT = 16'd89;
            4'd12:      LUT = 16'd144;
            4'd13:      LUT = 16'd233;
            4'd14:      LUT = 16'd377;
            4'd15:      LUT = 16'd610;
            default:    LUT = 16'hDEAD;
        endcase
    endfunction

    logic               calc_sel_in,
                        calc_rg_en,
                        clear_reg,
                        calc_sel_out,
                        calcselout_,
                        is_looping,
                        calc_done,
                        reg_overflow;
    logic [1:0]         fib_sel_out,
                        fibsel_;
    logic [OUTBITS-1:0] regC, regD;
    logic [INBITS-1:0]  n_captured,
                        n_,
                        i,
                        i_next;

    assign calc_done    = (fibsel_[0]) || (!is_looping);
    assign reg_overflow = (regC > FIBMAX) || (regD > FIBMAX);


    // ~~~~ module instantiations ~~~~

    logic [1:0][OUTBITS-1:0]mux_din_a,
                            mux_din_b,
                            mux_din_calc;
    logic [3:0][OUTBITS-1:0]mux_din_fib;
    logic [OUTBITS-1:0]     mux_dout_a,
                            mux_dout_b,
                            mux_dout_calc,
                            lut_lookup;

    assign mux_din_a    = {regC, {{(16){1'b0}}, 16'd377} };
    assign mux_din_b    = {regD, {{(16){1'b0}}, 16'd610} };
    assign mux_din_calc = {regD, regC};
    assign lut_lookup   = {16'h0000, LUT(n_captured[3:0])};
    assign mux_din_fib  = {ERR_CODE, ERR_CODE, lut_lookup, mux_dout_calc};

    Mux_N
    #(
        .WIDTH      (OUTBITS),
        .INPUTS     (2),
        .SEL_WIDTH  ()
    )
    mux_a
    (
        .DIN        (mux_din_a),
        .SEL        (calc_sel_in),
        .DOUT       (mux_dout_a)
    );

    Mux_N
    #(
        .WIDTH      (OUTBITS),
        .INPUTS     (2),
        .SEL_WIDTH  ()
    )
    mux_b
    (
        .DIN        (mux_din_b),
        .SEL        (calc_sel_in),
        .DOUT       (mux_dout_b)
    );

    Mux_N
    #(
        .WIDTH      (OUTBITS),
        .INPUTS     (2),
        .SEL_WIDTH  ()
    )
    mux_calc
    (
        .DIN        (mux_din_calc),
        .SEL        (calc_sel_out),
        .DOUT       (mux_dout_calc)
    );

    Mux_N
    #(
        .WIDTH      (OUTBITS),
```

```
        .INPUTS     (4),
        .SEL_WIDTH  ()
    )
    mux_fib
    (
        .DIN        (mux_din_fib),
        .SEL        (fib_sel_out),
        .DOUT       (fib_o)
    );

    logic [OUTBITS-1:0] c_sum, d_sum;
    logic               unused1, unused2;

    BKAdder
    #( .WIDTH(OUTBITS))
    AddrC
    (
        .a(mux_dout_a),
        .b(mux_dout_b),
        .cin(1'b0),
        .sum(c_sum),
        .cout(unused1)
    );

    BKAdder
    #( .WIDTH(OUTBITS))
    AddrD
    (
        .a(mux_dout_a),
        .b(mux_dout_b << 1),
        .cin(1'b0),
        .sum(d_sum),
        .cout(unused2)
    );


    // ~~~~ synch logic ~~~~
    always_ff @(posedge clk_i) begin

        // ~~ priority 0 : reset
        if(rstn_i == 1'b0 || reg_overflow) begin
            i           <= '0;
            n_captured  <= '0;
            fib_sel_out <= '0;
            regC        <= '0;
            regD        <= '0;
            CS          <= RESET;
        end
        // ~~ priority 1 : state wiggles ~~
        else begin
            i           <= i_next;
            n_captured  <= n_;
            fib_sel_out <= fibsel_;
            calc_sel_out<= calcselout_;

            // register update
            if(clear_reg) begin
                regC    <= '0;
                regD    <= '0;
            end
            else if(calc_rg_en) begin
                regC    <= c_sum;
                regD    <= d_sum;
            end

            CS          <= NS;
        end
    end



    // ~~~~ comb logic ~~~~
    always_comb begin

        // ~~ system defaults ~~
        rdy_in_o        = 1'b0;
        valid_o         = 1'b0;
        i_next          = '0;
        clear_reg       = 1'b0;
        n_              = n_captured;

        // ~ CALC defaults ~
        calc_sel_in     = 1'b0;
        calc_rg_en      = 1'b0;
        calcselout_     = calc_sel_out;
        fibsel_         = fib_sel_out;
        is_looping      = 1'b1; // start loop if not using LUT

        case(CS)

            RESET:
            begin
                calcselout_ = 1'b0;
                fibsel_     = 2'b00;
                clear_reg   = 1'b1;

                NS          = IMPORT;
            end

            IMPORT:
            begin
                calcselout_ = 1'b0;
                rdy_in_o    = 1'b1;
                fibsel_     = 2'b00;
                clear_reg   = 1'b1;

                if(valid_i) begin
                    n_          = n_i;
```

```
                    NS          = CALC;
                end
                else begin
                    NS          = IMPORT;
                end
            end

            CALC:
        begin
                // Priority 0 : n is inside LUT
                if(n_captured <= LUTMAX) begin
                    calcselout_ = 1'b0;
                    fibsel_     = 2'b01;
                    clear_reg   = 1'b1;
                end

                // Priority 1 : (edge case) fib will be greater than OUTBITS
                else if(n_captured > NMAX) begin
                    calcselout_ = 1'b0;
                    fibsel_     = 2'b11;
                    clear_reg   = 1'b1;
                end

                // Priority 2 : n needs to be calculated
                else begin
                    fibsel_     = 2'b00;
                    calc_rg_en  = 1'b1;

                    // Priority 2a : start calc loop
                    if(i == 0) begin
                        i_next      = LUTMAX + 8'h03;
                        calc_sel_in = 1'b0;

                        // n is 16 or 17
                        if((n_captured - (LUTMAX + 8'h01)) == 0 ) begin
                            calcselout_ = 1'b0;
                            is_looping  = 1'b0;

                        end
                        else if((n_captured - (LUTMAX + 8'h01)) == 1 ) begin
                            calcselout_ = 1'b1;
                            is_looping  = 1'b0;
                        end
                    end

                    // Priority 2b : continue loop
                    else begin
                        i_next      = i + 8'h02;
                        calc_sel_in = 1'b1;

                        if( (n_captured - i) == 0 ) begin
                            calcselout_ = 1'b0;
                            is_looping  = 1'b0;
                        end
                        else if((n_captured - i) == 1 ) begin
                            calcselout_ = 1'b1;
                            is_looping  = 1'b0;
                        end
                    end
                end

                if(calc_done === 1'b1)  NS = EXPORT;
                else                    NS = CALC;

            end

            EXPORT:
            begin
                valid_o     = 1'b1;
                if(rdy_out_i === 1'b1)  NS = IMPORT;
                else                    NS = EXPORT;

            end

            default:
            begin
                NS          = RESET;
            end
        endcase
    end

    `ifdef FORMAL

        // ========================
        // locals
        // ========================

        logic                   f_past_valid;
        initial                 f_past_valid = 0;
        always @(posedge clk_i) f_past_valid <= 1;
        always_comb             if(!f_past_valid) assume (!rstn_i);

        logic   f_waiting,
                f_reading,
                f_calcing,
                f_writing;

        assign  f_waiting = rdy_in_o && !valid_i;
        assign  f_reading = rdy_in_o && valid_i;
        assign  f_calcing = !rdy_in_o && !valid_o;
        assign  f_writing = !rdy_in_o && valid_o;

        // ========================
        // Fib Boundary Checks (TODO not passing Ind)
        // ========================

        // a zero output must mean the input was zero (not including reset)
        always @(posedge clk_i) begin
```

```
            if( (fib_o == 32'd0 && $rose(f_writing)) &&
                (rstn_i && $past(rstn_i) && $past(rstn_i, 2) && $past(rstn_i, 3)) &&
                (f_past_valid && $past(f_past_valid) && $past(f_past_valid, 2) && $past(f_past_valid, 3))
            ) begin
                assert($past(n_i, 2) == 8'd0);
                assert($past(f_reading, 2));
            end
        end


        // high values of n should take 2 cycles to output the ceil code
        always @(posedge clk_i) begin
            if(f_past_valid && (fib_o == ERR_CODE && $rose(f_writing))) begin
                assert($past(n_i,2) > NMAX);
                assert($past(f_reading, 2));
            end
        end


        // ========================
        // Data Integrity (TODO not passing Ind)
        // ========================
        function automatic longint unsigned fibboni(
            input logic [INBITS-1:0] n
        );
            longint unsigned a, b, t;
            a       = 0;
            b       = 1;
            for(int i = 0; i < NMAX; i++) begin
                if(i < n) begin
                    t = a + b;
                    a = b;
                    b = t;
                end
            end
            fibboni = a;
        endfunction

    `endif

endmodule
```

5.1.2 Simulation SVA Testing

Listing 5.4: Testbench simulation for Fibonacci Accelerator

```
`timescale 1ns / 1ps

module tb_fib();

    // ~~~~ imports ~~~~
    import tb_utils_pkg::*;

    // ~~~~ locals ~~~~
    localparam      CLK_PERIOD = 10;
    logic           clk,
                    resetn,
                    n_is_valid,
                    fib_was_read,
                    rdy_new_n,
                    fib_is_valid;

    logic [7:0]     n;
    logic [31:0]    fib_actual, fib_expected;
    localparam  logic [31:0]ERR_CODE    = 32'hFFFF_FFFF;
    localparam  logic [7:0] INBITS      = 8;
    localparam  logic [7:0] NMAX        = 8'd47;

    // ~~~~ functions ~~~~
    function automatic longint unsigned fibboni(
        input logic [INBITS-1:0] N
    );
        longint unsigned a, b, t;
        a       = 0;
        b       = 1;
        for(int unsigned i = 0; 8'(i) < NMAX; i++) begin
            if(i < N) begin
                t = a + b;
                a = b;
                b = t;
            end
        end
        fibboni = a;
    endfunction

    // ~~~~ tasks ~~~~
    task automatic request_fn(
        input logic [7:0]   n_val
    );
        // ~ IMPORT ~
        do @(posedge clk);
        while(rdy_new_n !== 1'b1);

        n               = n_val;
        n_is_valid      = 1'b1;
        @(posedge clk);

        // ~ CALC -> EXPORT ~
        do @(posedge clk);
        while(fib_is_valid !== 1'b1);
        n_is_valid      = 1'b0;
        @(posedge clk);
    endtask

    // ~~~~ instantiations ~~~~
    fib
    #(
```

```
        .INBITS     (8),
        .OUTBITS    (32)
    )
    Fib
    (
        .clk_i      (clk),
        .rstn_i     (resetn),
        .n_i        (n),
        .valid_i    (n_is_valid),
        .rdy_out_i  (fib_was_read),

        .rdy_in_o   (rdy_new_n),
        .fib_o      (fib_actual),
        .valid_o    (fib_is_valid)
    );

    // ~~~~ heartbeat ~~~~
    always #(CLK_PERIOD/2) clk = ~clk;


    // ~~~~ testing ~~~~
    testcase tc;
    logic [31:0] tnum;

    `ifndef NO_SVA
        // ~~ sequences (unsupported by icarus) ~~

        // ~~ properties (unsupported by icarus) ~~
        property p1;
            @(posedge clk) disable iff(!resetn)
                $rose(resetn) |-> rdy_new_n;
        endproperty

        property p2;
            @(posedge clk) disable iff(!resetn)
                not(rdy_new_n && fib_is_valid);
        endproperty

        property p3;
            @(posedge clk) disable iff(!resetn)
                ($past(fib_is_valid) && fib_is_valid && !fib_was_read) |->
                    ($past(fib_actual) == fib_actual);
        endproperty

        // ~~ concurrents (unsupported by icarus)~~
        reset_import    : assert property(p1) else tc.err("Didn't enter IMPORT after RESET");
        import_export   : assert property(p2) else tc.err("Shouldn't accept new data while holding old");
        hold_fib        : assert property(p3) else tc.err("fib output wasn't held");
    `endif

    // ~~~~ time-dependent testing ~~~~
    initial begin

        // ~~ waveforms ~~
        $dumpfile("tb_fib.vcd");
        $dumpvars(0);

        // ~~ defaults ~~
        tc              = new();
        clk             = 1'b1;
        resetn          = 1'b0;
        n               = 8'h00;
        n_is_valid      = 1'b0;
        fib_was_read    = 1'b0;

        // ~~ fresh start ~~
        #100;
        resetn          = 1'b1;

        // ~~ immediate ~~
        tc.new_test("n in [0,47]");
        tnum = tc.get_testnum();

            for(logic [7:0] i = 8'h00; i < 8'd48; i = i + 1'b1) begin

                request_fn(.n_val(i));
                fib_expected = 32'(fibboni(n));
                assert(fib_expected === fib_actual) else begin tc.err("fib value didnt match");end

                repeat(2)begin @(posedge clk); end
                fib_was_read    = 1'b1;
                @(posedge clk);
                fib_was_read    = 1'b0;
            end
            @(posedge clk);
        tc.test_done;

        tc.new_test("n in 48+");
        tnum = tc.get_testnum();

            for(logic [7:0] i = 8'd48; i < 8'hFF; i = i + 1'b1) begin

                request_fn(.n_val(i));
                assert(fib_actual == ERR_CODE) else begin tc.err("fib value didnt match"); end

                repeat(2)begin @(posedge clk); end
                fib_was_read    = 1'b1;
                @(posedge clk);
                fib_was_read    = 1'b0;
            @(posedge clk);
            end
        tc.test_done;

        $finish();
    end

endmodule
```

5.2 Boneyard

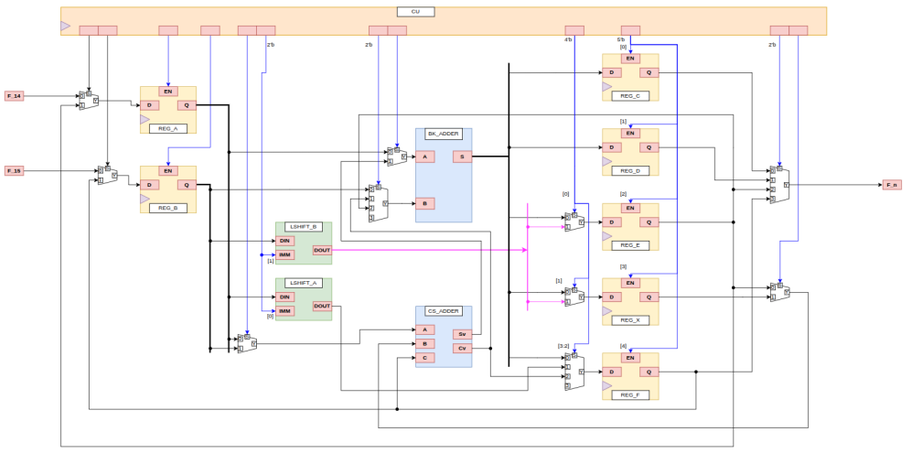


Figure 5.1: First Draft of Accelerator

Chapter 6

References

1. CPE490 - OpenSource ASIC